LABORATORY ASSIGNMENT

Course	Full Stack Web Development
Course Code	23CM4121
Experiment No.	1
Student Name	B.Jatin
Roll No.	A24126552070
Date	30-08-2026

1. TITLE

Implementation of Object-Oriented Inheritance Types in JavaScript (Single, Multilevel, Hierarchical, Multiple, and Hybrid)

2. AIM

To design and implement Object-Oriented Programming (OOP) inheritance models in JavaScript using ES6 classes, the **extends** keyword, and mixin techniques.

3. OBJECTIVE

• Understand OOP concepts in JavaScript ES6+.

• Implement Single Inheritance.

• Implement Multilevel Inheritance.

• Implement Hierarchical Inheritance.

• Demonstrate Multiple Inheritance using a mixin-style technique.

• Demonstrate Hybrid Inheritance by combining inheritance structures.

4. THEORY

Inheritance allows a derived class (child class) to acquire methods and properties from a base class (parent class), promoting code reuse and structured design. In JavaScript ES6, inheritance is commonly implemented with **class** and **extends**.

Single: one child inherits from one parent.

Multilevel: inheritance forms a chain across multiple class levels.

Hierarchical: multiple children inherit from one common parent.

Multiple: features from more than one source are combined; JavaScript classes do not support multiple **extends** directly.

Hybrid: a combination of two or more inheritance structures.

Application Flow: Base Class → Child Class / Combined Methods → Object Instantiation → Method Execution → Output

5. ALGORITHM / PROCEDURE

1. Create an assignments/inheritencetypes directory.

2. Create single.js and implement Bird → Parrot.

3. Create multilevel.js and implement Tree → FruitTree → MangoTree.

4. Create hierarchical.js and implement Instrument → Guitar and Piano.

5. Create multiple.js and combine Father and Mother behavior in Child.

6. Create hybrid.js using Grandparent → Parent → Child1/Child2.

7. Execute each file with Node.js and verify the output.

6. PROGRAM

single.js — SINGLE INHERITANCE

```
class Bird {     fly() {
console.log("Bird can fly");     } }
```

```
class Parrot extends Bird {     speak()
{         console.log("Parrot can
speak");     } }

let p = new Parrot();
p.fly();
p.speak();
```

Expected Output:
Bird can fly
Parrot can speak

multilevel.js — MULTILEVEL INHERITANCE

```
class Tree {     grow() {
console.log("Tree can grow");     } }

class FruitTree extends Tree {     fruit() {
console.log("Fruit tree gives fruit");     } }

class MangoTree extends FruitTree {     mango()
{         console.log("Mango tree gives
mangoes");     } }

let m = new MangoTree();
m.grow();
m.fruit();
m.mango();
```

Expected Output:
Tree can grow
Fruit tree gives fruit
Mango tree gives mangoes

hierarchical.js — HIERARCHICAL INHERITANCE

```
class Instrument {     play() {
console.log("Instrument can play");     } }

class Guitar extends Instrument {
strum() {         console.log("Guitar is
strumming");     } }

class Piano extends Instrument {
press() {         console.log("Piano is
playing");     } }

let g = new Guitar();
let p = new Piano();
g.play();
g.strum();
p.play();
p.press();
```

Expected Output:
Instrument can play
Guitar is strumming
Instrument can play
Piano is playing

multiple.js — MULTIPLE INHERITANCE

```
class Father {     fatherProperty() {
console.log("Father's property");     } }

class Mother {     motherProperty() {
console.log("Mother's property");     } }

class Child extends Father {
motherProperty() {
console.log("Mother's property");     } }

let child = new Child();
child.fatherProperty();
child.motherProperty();
```

Expected Output:
Father's property
Mother's property

hybrid.js — HYBRID INHERITANCE

```
class Grandparent {     showGrandparent() {
console.log("This is Grandparent");     } }

class Parent extends Grandparent {
showParent() {
console.log("This is Parent");     } }

class Child1 extends Parent {
showChild1() {
console.log("This is Child 1");     } }

class Child2 extends Parent {
showChild2() {
console.log("This is Child 2");     } }

let child1 = new Child1();
let child2 = new Child2();

child1.showGrandparent();
child1.showParent();
child1.showChild1();

child2.showGrandparent();
child2.showParent();
child2.showChild2();
```

Expected Output:
This is Grandparent
This is Parent
This is Child 1
This is Grandparent
This is Parent
This is Child 2

7. CODE EXPLANATION

let p = new Parrot();	Creates an object of the derived class.
MangoTree extends FruitTree	Forms a multilevel inheritance chain.
Guitar extends Instrument	Shows hierarchical inheritance with a common parent.
Child extends Father	
Child1/Child2 extends Parent	Combines multilevel and hierarchical structures for hybrid inheritance.

Code / Syntax	Explanation
class Child extends Parent	Creates a derived class that inherits from Parent.

Uses Father as the direct parent and adds Mother's method to simulate multiple inheritance in JavaScript.

8. TEST CASES AND EXECUTION DATA

Test Case	Script / Input	Expected Output	Status
TC-01	node single.js	Bird can fly; Parrot can speak	To Verify
TC-02	node multilevel.js	Tree can grow; Fruit tree gives fruit; Mango tree gives mangoes	To Verify
TC-03	node hierarchical.js	Instrument can play; Guitar is strumming; Instrument can play; Piano is playi	ngTo Verify
TC-04	node multiple.js	Father's property; Mother's property	To Verify
TC-05	node hybrid.js	This is Grandparent; This is Parent; This is Child 1; This is Grandparent; T Child 2	To Verify

9. OUTPUT

Output 1: single.js

```
Bird can fly
Parrot can speak
```

Output 2: multilevel.js

```
Tree can grow
Fruit tree gives fruit
Mango tree gives mangoes
```

Output 3: hierarchical.js

```
Instrument can play
Guitar is strumming
Instrument can play
Piano is playing
```

Output 4: multiple.js

```
Father's property
Mother's property
```

Output 5: hybrid.js

```
This is Grandparent
This is Parent
This is Child 1
This is Grandparent
This is Parent
This is Child 2
```

10. RESULT

Thus, Single, Multilevel, Hierarchical, Multiple, and Hybrid inheritance were implemented using JavaScript classes and related techniques. The programs can be executed in Node.js and verified against the expected outputs.